

Vending Machine Electronics Instructions

1. Overview

This document explains the electronics used in a DIY vending machine. You do **not** need to be an electronics expert to follow this but ideally some competency with Arduino is required. This document isn't a definitive guide, you can wire this however you want this is just an idea of how I approach it.

The vending machine electronics are based around:

- **12 V DC power** for motors and solenoids
- An **Arduino Nano** as the main controller, another Arduino Nano used to control the motors.
- Modular wiring so parts can be built and tested independently

At the time of release, you have to make the boards yourself. It could be breadboard or made with proto board. I've provided gerber files but these are currently untested. I will be creating these as a kit of parts in the future.

2. High-Level System Layout

To keep things simple and reliable, the machine is split into **two main electronics boards**:

2.1 Motor Control Board (Back of Machine)

This board is responsible only for **dispensing items**.

It contains:

- Arduino Nano
- Stepper motor drivers (DRV8825)
- Connectors for NEMA 17 stepper motors
- 12 V power input

This board:

- Does **not** handle user input
- Does **not** control the display
- Only receives simple commands like: *"Dispense motor 3"*

2.2 Control / Interface Board (Front of Machine)

This board handles everything the user interacts with:

- Keypad (button matrix)
- Screen (I2C)
- Door solenoid
- Coin counter

It also contains:

- Arduino Nano

The two boards communicate using **serial (TX/RX)**.

Why split it this way?

- Easier to wire
- Easier to debug
- Less electrical noise from motors near sensitive inputs
- You can upgrade or replace one board without touching the other

2.3 Alternative

You could get a larger Arduino like a ESP32 or Arduino Mega and wire everything to it! It'll take more wiring, but remove the complexity of the serial comms between 2 boards.

3. Power System (12 V Everywhere)

3.1 Why 12V?

12 V is recommended for the entire machine because:

- Stepper motors work well at 12 V
- Door latch solenoids are commonly 12 V
- Power supplies are cheap and widely available

3.2 Logic Power

- The Arduino Nano is powered from 12 V via its **VIN** pin. All grounds (GND) are connected together
 - Most Arduino Nanos run on 5V logic. Do not connect 12V to any arduino pin or it will break it instantly.
-

4. Controlling Stepper Motors (NEMA 17 + DRV8825)

Every dispensing mechanism in the machine uses a **NEMA 17 stepper motor**.

4.1 What is a Stepper Motor?

A stepper motor moves in **small precise steps** instead of spinning freely like a DC motor. This makes it ideal for vending machines because:

- You can control exact movement
- No position sensors are required
- Motors stop holding position when powered

4.2 Motor Type

- **NEMA 17 bipolar stepper motors**
- 4-wire motors

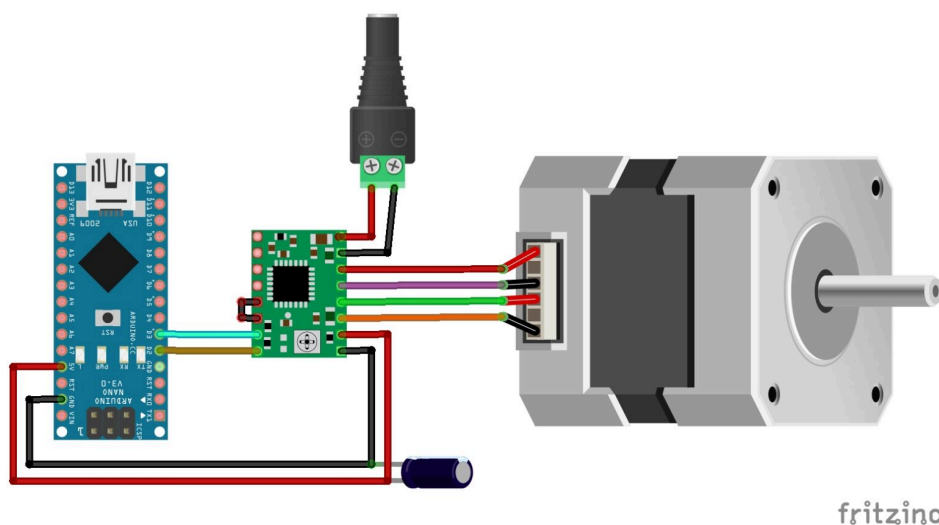
⚠ Always check the **wire colour pairs** – different manufacturers use different colours.

4.3 DRV8825 Stepper Driver (Standard Wiring)

The DRV8825 is a popular stepper driver module.

Normally it is wired with:

- STEP pin → Arduino
- DIR pin → Arduino
- ENABLE pin → Arduino (optional)
- Microstep pins (M0/M1/M2)



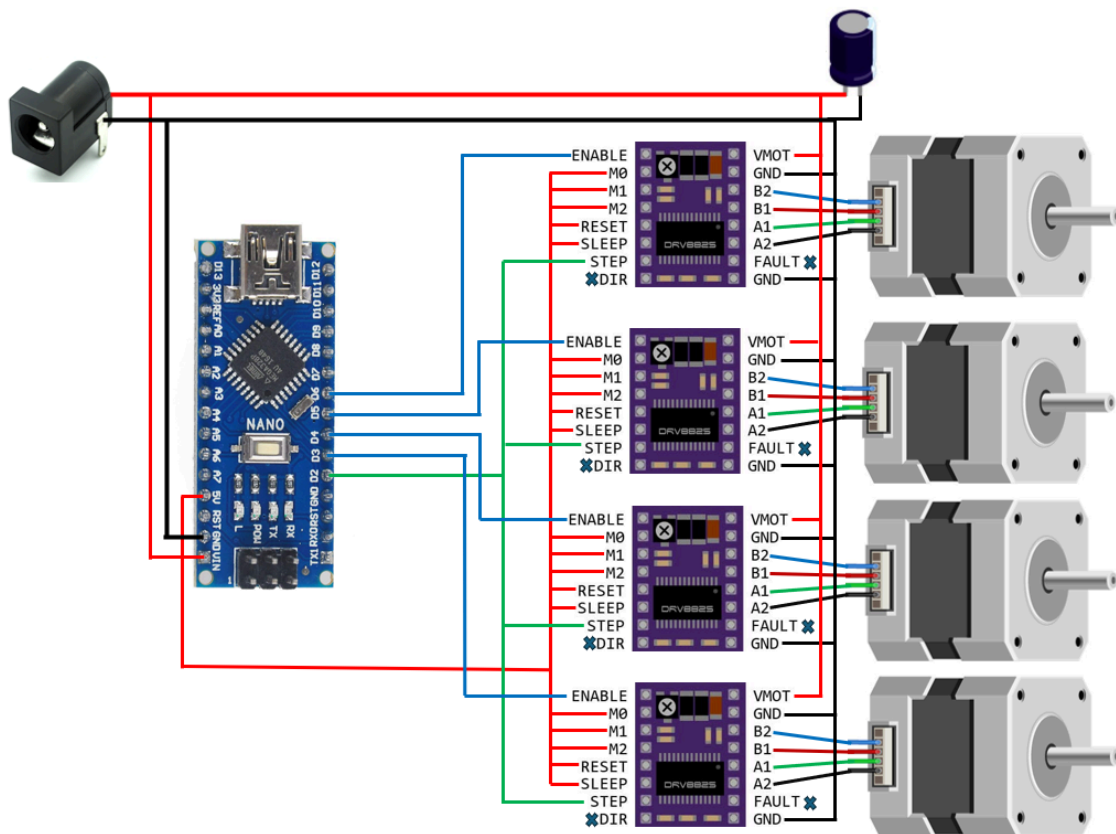
4.4 Simplified "Cheat" Wiring Method (Recommended)

To reduce wiring complexity:

- All **STEP** pins are tied together
- **DIR** pins are not used (single direction only)
- Each motor has its own **ENABLE** pin

This means:

- One Arduino pin generates step pulses for *all* motors
- Individual motors are selected by enabling one driver at a time
- You'll only need 1 pin per motor, plus 1 pin for the step signal. Eg 16 motors = 17 pins.



4.5 Microstepping

Microstepping allows smoother motion and quieter motors.

It is controlled using the **M0** / **M1** / **M2** pins on the DRV8825.

Example:

- M0 = HIGH
- M1 = HIGH
- M2 = HIGH

This gives **1/32 microstepping**.

⚠ At 1/32 microstepping:

- One full motor rotation = **6400 steps** (instead of 200)

Higher microstepping:

- Smoother motion
- Less vibration
- Slightly reduced torque

4.6 Powering the Motors

- Motor supply: **12 V**
- Set the current limit on each DRV8825 **before connecting motors**

📺 *How to set output current limit on A4988 stepper driver*

I set each motor to 1.2A, so VREF = 0.6V on the multimeter.

⚠ Never plug or unplug stepper motors while powered – it can destroy the driver.

4.7 Controlling Steppers in Code (AccelStepper)

To simplify motion control, the **AccelStepper** Arduino library is used.

Basic example (single shared STEP pin, individual ENABLE pins):

```
#include <AccelStepper.h>
```

```
#define STEP_PIN 2
```

```
#define ENABLE_A1 3
```

```
AccelStepper stepper(AccelStepper::DRIVER, STEP_PIN, STEP_PIN);
```

```
void setup() {
  pinMode(ENABLE_A1, OUTPUT);
  digitalWrite(ENABLE_A1, LOW); // enable motor
```

```
  stepper.setMaxSpeed(2000);
  stepper.setAcceleration(1000);
```

```
}
```

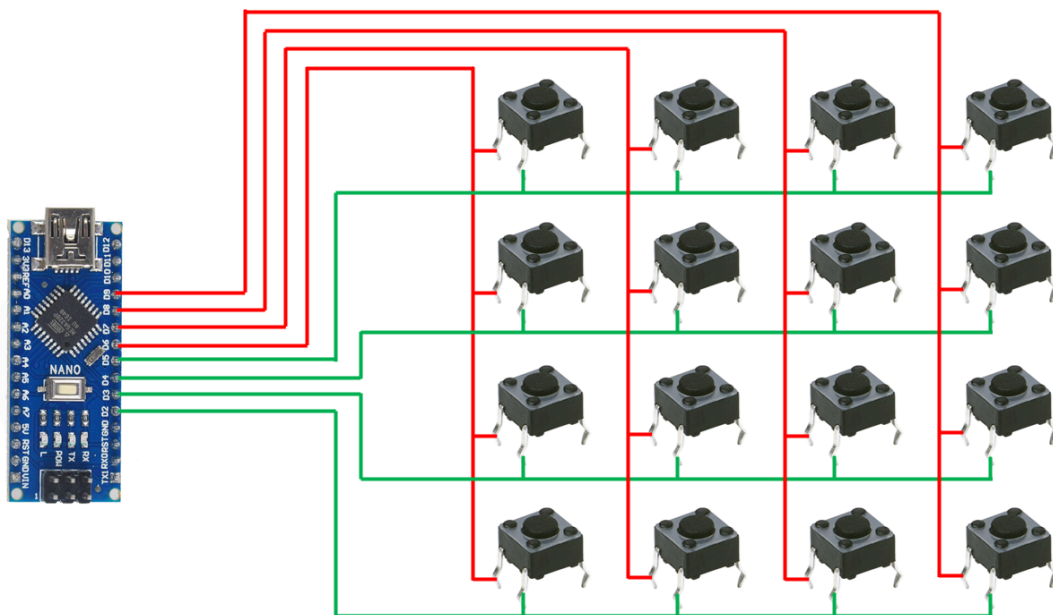
```
void loop() {  
  stepper.move(6400); // one full rotation at 1/32 microstepping  
  stepper.runToPosition();  
  
  digitalWrite(ENABLE_A1, HIGH); // disable motor after dispense  
  delay(2000);  
}
```

Only **one motor is enabled at a time**, ensuring only the selected spiral turns.

5. Keypad (Button Matrix)

To reduce wiring, the keypad is wired as a **matrix** (rows and columns) instead of one wire per button. This reduces the pins required from 16 to 8, and adding another row only adds 1 more pin.

5.1 Wiring (Example)



5.2 Code Snippet (Keypad Library)

This is the simplest way for beginners.

```
#include <Keypad.h>

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {2, 3, 4, 5};    // wire to the row pins
byte colPins[COLS] = {6, 7, 8, 9};    // wire to the column pins

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

void setup() {
  Serial.begin(115200);
}

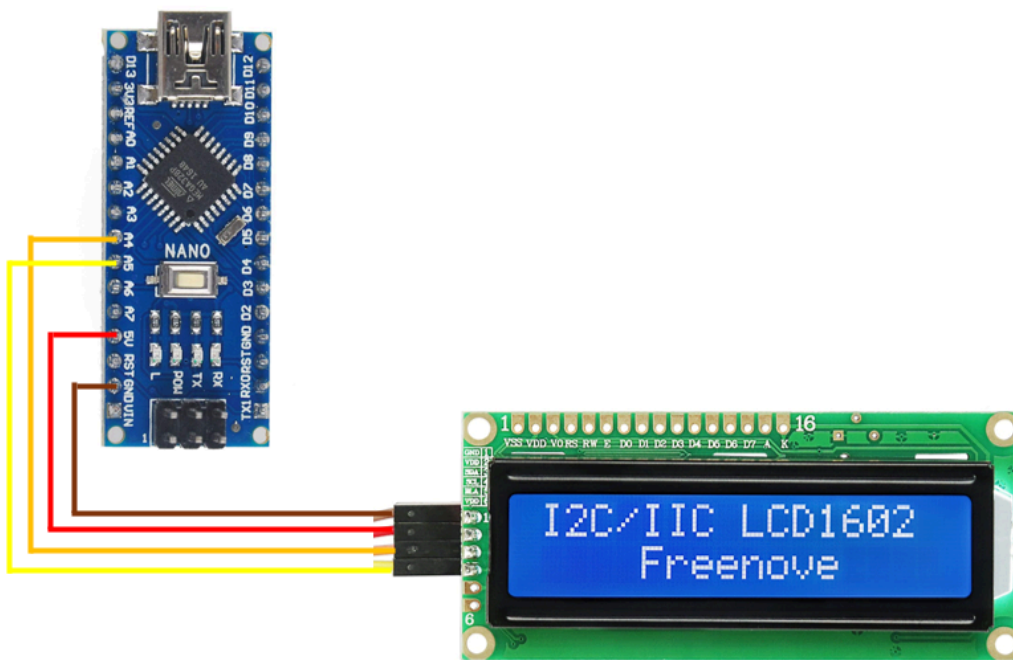
void loop() {
  char key = keypad.getKey();
  if (key) {
    Serial.print("Key pressed: ");
    Serial.println(key);
  }
}
```

6. Screen (I2C)

The screen uses **I2C** which keeps wiring simple (only 4 wires).

6.1 Wiring

- **VCC** → 5 V (or 3.3 V depending on your display)
- **GND** → GND
- **SDA** → Arduino SDA
- **SCL** → Arduino SCL



6.2 Code Snippet (I2C LCD Example)

For the common 16x2 / 20x4 I2C LCD backpacks:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // address may be 0x27 or 0x3F
```

```
void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Vending Machine");
  lcd.setCursor(0, 1);
  lcd.print("Insert coin...");
}
```

```
void loop() {
}
```

Tip: If nothing shows, your I2C address is probably different. Use an “I2C scanner” sketch to find it.

7. Door Lock Solenoid (12 V)

The door lock uses a **12 V DC solenoid** (sometimes called a door latch / door strike).

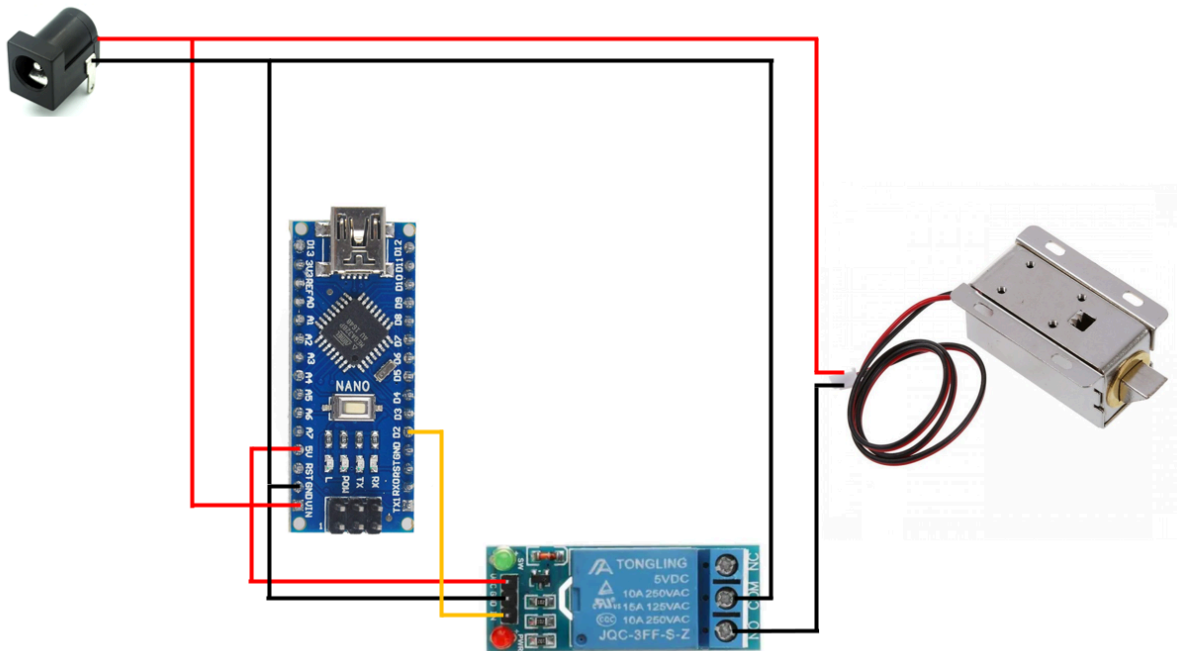
7.1 Why a Solenoid?

- Simple
- Reliable
- Strong holding force

7.2 Driving the Solenoid (Important)

Solenoids are **inductive loads** and must **never** be driven directly from an Arduino pin.

Use a breakout relay board which lets you put logic (5v) in, and switch the high current solenoid.



7.3 Code Snippet (Solenoid Output)

This assumes your driver circuit makes the solenoid turn on when the Arduino pin is HIGH.

```
const int SOLENOID_PIN = 10;
```

```
void setup() {  
  pinMode(SOLENOID_PIN, OUTPUT);  
  digitalWrite(SOLENOID_PIN, LOW);  
}
```

```
void loop() {
```

```
// Example: unlock for 500ms
digitalWrite(SOLENOID_PIN, HIGH);
delay(500);
digitalWrite(SOLENOID_PIN, LOW);

delay(3000);
}
```

8. Coin Counter

Most coin acceptors output **pulses** (a short on/off signal) for each valid coin.

- Each pulse = one coin event
- Some acceptors output different pulse counts per coin value (e.g. £1 = 1 pulse, £2 = 2 pulses)

8.1 Wiring

- Coin acceptor **signal output** → Arduino digital input
- **Common GND** between coin acceptor and Arduino

8.2 Code Snippet (Interrupt Pulse Counting)

This example counts pulses using an interrupt and a simple debounce.

```
const int COIN_PIN = 2; // use an interrupt-capable pin on Nano (D2 or D3)
```

```
volatile unsigned long coinPulses = 0;
volatile unsigned long lastPulseMs = 0;
```

```
void onCoinPulse() {
  unsigned long now = millis();
  if (now - lastPulseMs > 20) { // debounce (adjust to suit)
    coinPulses++;
    lastPulseMs = now;
  }
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(COIN_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(COIN_PIN), onCoinPulse, FALLING);
}
```

```
void loop() {
```

```
static unsigned long lastPrint = 0;

if (millis() - lastPrint > 500) {
  lastPrint = millis();

  noInterrupts();
  unsigned long p = coinPulses;
  interrupts();

  Serial.print("Coin pulses: ");
  Serial.println(p);
}
}
```

9. Serial Communication Between Boards

The two Arduino boards communicate using **UART serial**.

- TX → RX
- RX → TX
- Common GND

Example messages:

- DISPENSE A3
- DONE A3
- ERROR JAM A3

This keeps the system modular and easy to debug.

10. Final Notes for Beginners

- Build and test **one section at a time**
- Start with motors only
- Then keypad + screen
- Then solenoid
- Then coin acceptor

If something doesn't work:

- Check power first
- Then wiring
- Then software